

PROJECT AION

Technical Manual

Multi-Archon Autonomous Operations Platform

Jarvis (Master Archon) + Alfred-Pro (Operations Archon)

Generated: 2026-03-29 10:59

Version 1.0.0

Jarvis v5.11.0 | Alfred-Pro v3.2.0 | Pulse v1.0.0-aion

Table of Contents

1. Project Aion Overview
 2. Architecture
 3. Jarvis — Master Archon
 4. Alfred-Pro — Operations Archon
 5. Pulse — Shared Task Management
 6. Shared Infrastructure
 7. Nexus — Headless Job Orchestration
 8. Integration Layer
 9. Security Architecture
 10. Operational Procedures
-

1. Project Aion Overview

Vision

Project Aion is a multi-Archon autonomous operations platform. It combines specialized AI agents ("Archons") into a cooperative ecosystem that handles software development, infrastructure management, task orchestration, and knowledge management — all running on a single Mac Studio.

The core insight: different types of work require different operational models. Deep coding sessions need persistent context and memory. Infrastructure health checks need always-on stateless monitoring. Task management needs a shared database accessible to all agents. By splitting these concerns across specialized Archons, each can excel at its domain.

Current Archons

Archon	Role	Operational Model	Status
Jarvis	Master Archon — deep collaborative work	Persistent tmux sessions, JICM context management, 4-tier memory	Active (v5.11.0)
Alfred-Pro	Operations Archon — always-on automation	Cron-driven dispatcher, headless Claude Code, persona isolation	Active (v3.2.0)
Pulse	Shared task management bus	FastAPI REST service, PostgreSQL-backed, launchd 24/7	Active (v1.0.0-aion)

Planned Archons

Archon	Role	Based On
Sentinel	Security monitoring, threat detection, audit	Alfred security guards + expanded scope
Scholar	Deep research, knowledge graph curation, content generation	Jarvis research-ops + Loom pipeline
Artificer	Docker deployment, VM orchestration, CI/CD	Alfred Infra Deployer + Jarvis docker-deployer

Host Environment

Property	Value
----------	-------

Hardware	Mac Studio (Apple Silicon)
OS	macOS Darwin 25.2.0
Shell	zsh (default), bash 5.3.9 via MacPorts
Python	3.12+
Docker	Docker Desktop for Mac
tmux	Custom build at /Users/nathanielcannon/bin/tmux

2. Architecture

Layer Model

Project Aion uses a three-layer architecture (Greek terminology from the Archon model):

Layer	Greek	Purpose	Location
Nous	Knowledge	Patterns, state, priorities, context	.claude/context/
Pneuma	Capabilities	Skills, hooks, agents, commands, scripts	.claude/
Soma	Infrastructure	Docker, scripts, interfaces, hardware	/Jarvis/, /AIFred-Pro/

A fourth concept, **Neuro** (navigation substrate), represents the cross-references and pathways connecting layers. **Psyche** (topology maps) documents the Neuro.

Communication Channels

Archons communicate through shared infrastructure, not direct message passing:

Channel	Purpose	Technology
Pulse API	Task creation, status updates, labels	REST API on port 8700
PostgreSQL	Shared state (tasks, sessions, analytics)	Port 5432, multiple databases
Qdrant	Semantic search (embeddings)	Port 6333, shared collections
Neo4j	Knowledge graph	Port 7474, group_id per Archon
Redis	Cache, working memory	Port 6379
Signal files	Real-time coordination	.claude/context/ flag files
Telegram	Async human notification	@KeryxArchon_bot
tmux	Direct keystroke injection (Jarvis only)	Jarvis tmux session

Repository Layout

Repository	Path	Purpose
------------	------	---------

Jarvis	/Users/nathanielcannon/Claude/Jarvis	Master Archon workspace
Alfred-Pro	/Users/nathanielcannon/Claude/AIFred-Pro	Operations Archon workspace
DwarfCron	/Users/nathanielcannon/Claude/Projects/DwarfCron	Chronicler product code
Alfred (baseline)	/Users/nathanielcannon/Claude/Alfred	Open-source baseline (read-only)

3. Jarvis — Master Archon

Identity

Jarvis is the Master Archon of Project Aion — a deeply autonomous, self-improving AI agent focused on collaborative coding, analysis, and project management. Personality: calm, precise, safety-conscious orchestrator with dry humor. "Butler precision + lab partner warmth + senior engineer competence."

Version: 5.11.0

Branch: Project_Aion (diverged from Alfred baseline at commit 2ea4e8b)

Autonomic Components (Hippocrenae)

Jarvis has 9 autonomic components (AC-01 through AC-09) plus one hidden override (AC-10):

AC	Name	Trigger	Purpose
AC-01	Self-Launch	Session start	Load identity, read state, greet, begin work
AC-02	Wiggum Loop	Always (DEFAULT)	Multi-pass verification: Execute → Check → Review → Drift Check → Context Check
AC-03	Milestone Review	Work completion	Independent quality gate with code-review + project-manager agents
AC-04	JICM	Context >= 55%	Intelligent context management: compress → /clear → resume
AC-05	Self-Reflection	Session end, /reflect	Analyze corrections, identify patterns, generate proposals
AC-06	Self-Evolution	/evolve, idle time	Implement queued improvements (risk-gated: low=auto, medium=notify, high=approve)
AC-07	R&D Cycles	/research	Research external innovations, internal efficiency
AC-08	Maintenance	/maintain	Health checks, freshness audits, log rotation
AC-09	Session Completion	/end-session	Update state, commit, push, trigger reflection

AC-10	Ulfhedthnar	Defeat signals >= 7	Berserker override: parallel agents, approach rotation. Auto-disengages.
-------	-------------	---------------------	--

JICM — Context Management (AC-04)

JICM (Jarvis Intelligent Context Management) is the most critical autonomic component. It prevents context window exhaustion through an external watcher process.

Threshold Architecture:

- 55%: JICM trigger (configurable via --threshold)
- 70%: Native auto-compact (CLAUDE_AUTOCOMPACT_PCT_OVERRIDE)
- 73%: Emergency /compact
- ~78.5%: Hard lockout ceiling

Two-Mechanism Resume:

1. Hook injection: session-start.sh reads .compressed-context-ready.md, injects as additionalContext
2. Idle-hands: Watcher detects flag, sends resume prompt via tmux keystrokes

Key Files:

- Watcher: .claude/scripts/jicm-watcher.sh (tmux window 1)
- Compression: .claude/agents/compression-agent.md
- Hook: .claude/hooks/session-start.sh
- Spec: .claude/context/components/AC-04-jicm.md

Memory Hierarchy (4-Tier)

Tier	Name	Latency	Storage	Purpose
0	Scratchpad	0s (force-loaded)	.claude/context/.scratchpad.md	Transient session details
1	MEMORY.md	0s (force-loaded)	~/.claude/projects/.../memory/MEMORY.md	Stable cross-session facts
2	Qdrant RAG	~2-3s	Vector DB (port 6333)	Semantic search across 4 collections
3	Graphiti KG	~20-30s	Neo4j (port 7474)	Knowledge graph with temporal awareness

Qdrant Collections: jarvis-context, codebase, research, sessions (all 2560-dim Cosine, Qwen3-Embedding-4B)

Aion Quartet (Infrastructure Processes)

Four always-on tmux processes that support Jarvis:

Window	Name	Role	Script
W1	Watcher	JICM monitoring, compression triggers	jicm-watcher.sh
W2	Ennoia	Session orchestrator, wake-up recommendations	ennoia.sh
W3	Virgil	Task/agent/file tracking	virgil.sh
W4	Commands	Signal-based command injection	command-handler.sh

tmux Session Layout

Window	Name	Purpose
W0	Jarvis	Primary Claude Code session (deterministic UUID: 17612316-37f1-5cec-b456-6a79f7735a9f)
W1	Watcher	JICM v7.1.1 context monitor (stop-and-wait architecture)
W2	Ennoia	Session orchestrator (intent-driven wake-up)
W3	Virgil	Task/agent/file change tracking
W4	Commands	Signal file → tmux keystroke injection
W5	Jarvis-dev	Dev/test session (--dev flag only)
W6+	Services	MLX-Embed, LiteLLM, LegendsViewer (auto-created)

Launch Modes:

- Default: Full Jarvis with resume by deterministic UUID
- --dev: Add W5 dev test driver
- --fresh: New session (archive old state)
- --lite: Isolated one-off (separate tmux session, cleaned on exit)
- --iterm2: iTerm2 native tabs via tmux -CC

JICM v7 Context Preparation

The v7 architecture replaced the 210-second LLM compression agent with a 0.06-second bash script (jicm-prep-context.sh). This script deterministically extracts active tasks, recent conversation, and priorities into

a checkpoint file — no LLM inference needed.

State Machine: WATCHING → HALTING → COMPRESSING → CLEARING → RESTORING → WATCHING

Parameter	Default	Purpose
JICM_TOKEN_THRESHOLD	300,000	Absolute token trigger
POLL_INTERVAL	5s	Statusline check frequency
COOLDOWN_PERIOD	600s	Min wait before next cycle
MAX_CONTEXT_TOKENS	1,000,000	Hard ceiling

Hooks

Jarvis hooks run within Claude Code's hook framework. They fire on lifecycle events and provide context injection, safety guards, and telemetry.

Hook	Event	Purpose
session-start.sh	SessionStart	Context injection, JICM resume, Alfred sync reminder
user-prompt-submit.js	UserPromptSubmit	Orchestration detection, context reminders
insight-capture.js	PostToolUse	Auto-capture session insights
virgil-tracker.js	PostToolUse	Track tasks, agents, file changes
ulfhedthnar-detector.js	PostToolUse	Detect defeat signals for AC-10
jicm-continuation-verifier.js	UserPromptSubmit	Cascade reinforcement for JICM resume
branch-protection.js	PreToolUse	Prevent accidental main branch edits
precompact-analyzer.js	PreCompact	Checkpoint before compaction

Skills (28 registered)

Skill	Purpose
filesystem-ops	File CRUD operations
git-ops	Git via bash (replaces git MCP)
web-fetch	URL fetching, web search
weather	Weather via wttr.in
doc-ops	Word, Excel, PDF, PowerPoint
self-ops	Self-improvement, status, validation
mcp-ops	MCP/skill lifecycle

autonom-ops	Session orchestration, commands, JICM
dev-ops	W5→W0 testing
research-ops	Multi-source research (15 backends)
knowledge-ops	4-tier memory hierarchy operations
deck-ops	Slide presentations
pulse-ops	Shared Pulse task management
chronicler-ops	Dwarf Fortress / Chronicler operations
ulfhedthnar	Berserker override (locked)

Agents (12 active)

Agent	Model	Purpose
code-analyzer	Sonnet	Understand codebase structure
code-implementer	Sonnet	Write/modify/refactor code
code-review	Sonnet	Technical quality review
code-tester	Sonnet	Run tests, Playwright, screenshots
deep-research	Opus	Thorough research with citations
docker-deployer	Sonnet	Deploy Docker services
service-troubleshooter	Sonnet	Diagnose infra issues
project-manager	Sonnet	Progress/alignment review
compression-agent	Haiku	JICM context compression
context-compressor	Haiku	Conversation compression
memory-bank-synchronizer	Sonnet	Sync docs with code changes
jicm-agent	Haiku	Autonomous JICM monitoring

Commands (40+)

Commands are slash-command specifications in `.claude/commands/*.md`. Key categories:

Category	Commands
Session	<code>/setup</code> , <code>/end-session</code> , <code>/checkpoint</code> , <code>/jicm</code>
Self-Improvement	<code>/reflect</code> , <code>/evolve</code> , <code>/research</code> , <code>/maintain</code>
Validation	<code>/tooling-health</code> , <code>/design-review</code> , <code>/validate-selection</code>
Context	<code>/context-budget</code> , <code>/smart-compact</code> , <code>/intelligent-compress</code>
Operations	<code>/jarvis-status</code> , <code>/usage</code> , <code>/health-report</code>
Development	<code>/dev-test</code> , <code>/analyze-codebase</code>

Orchestration	/orchestration:plan, /orchestration:status
---------------	--

MCP Servers

MCP	Purpose	Port/Transport
jarvis-rag	Semantic search (Qdrant)	stdio
jarvis-graphiti	Knowledge graph (Neo4j)	stdio
jarvis-pulse	Task management (Pulse API)	stdio
local-rag	Local document indexing	stdio
neo4j	Direct Neo4j queries	stdio

Patterns Library (51 patterns)

Jarvis maintains 51 behavioral patterns organized by category. The 6 mandatory patterns (must always be applied):

Pattern	Purpose
wiggum-loop-pattern	Multi-pass verification (DEFAULT for all tasks)
milestone-review-pattern	Quality gate at milestones
selection-intelligence-guide	Tool/agent/skill selection
jicm-pattern	Context management thresholds
startup-protocol	Session start sequence
session-exit	Clean session ending

4. Alfred-Pro — Operations Archon

Identity

Alfred-Pro is the Operations Archon — an always-on infrastructure automation platform that schedules headless AI jobs, manages task lifecycle, communicates via Telegram, and learns from feedback.

Version: 3.2.0

License: Proprietary (Dave O'Neil)

Architecture

Alfred-Pro uses a layered system of profiles, hooks, commands, and skills:

Profiles: Composable YAML layers that determine active hooks, permissions, patterns, and agents.

- `general.yaml` — Core hooks, base permissions (always active)
- `homelab.yaml` — Docker, NAS, monitoring
- `development.yaml` — Code projects, CI/CD
- `production.yaml` — Security hardening

Profile merging: Last layer wins (hooks), Union (patterns/skills/agents), Deny precedence (permissions)

CLAUDE.md Hard Gate

Alfred-Pro's CLAUDE.md includes a hard gate that blocks ALL work until infrastructure passes:

1. Docker running
2. Compose V2 available
3. `.env` exists with `PULSE_DB_PASSWORD`
4. PostgreSQL healthy
5. Pulse API responding

This ensures operational readiness before any Claude Code session begins.

Hooks (28 active + shared library)

Hook	Event	Purpose
------	-------	---------

session-start.js	SessionStart	Context setup, hard gate, versioning
prompt-dispatcher.js	UserPromptSubmit	Tool guidance (LSP, Docker, Git) + project detection + task routing
skill-router.js	UserPromptSubmit	Skill matching and invocation
orchestration-detector.js	UserPromptSubmit	Complexity scoring, auto-orchestration
planning-mode-detector.js	UserPromptSubmit	Plan mode detection
audit-logger.js	PreToolUse (*)	Full audit trail
secret-scanner.js	PreToolUse (Bash)	Credential leak prevention
branch-protection.js	PreToolUse (Bash)	Default branch protection
docker-validator.js	PreToolUse (Bash)	Docker command safety
document-guard.js	PreToolUse (Edit/Write)	4-tier file protection
credential-guard.js	PreToolUse	Pattern-based secret detection
file-access-tracker.js	PostToolUse	File access logging
cross-project-commit-tracker.js	PostToolUse	Multi-repo commit tracking
doc-sync-trigger.js	PostToolUse	Documentation sync
memory-maintenance.js	PostToolUse	Memory cleanup
docker-health-check.js	PostToolUse (Bash)	Container health monitoring
session-stop.js	Stop	Session cleanup
subagent-dispatcher.js	SubagentStop	Metrics + activity logging + chaining
pre-compact.js	PreCompact	Checkpoint before compaction
lib/shared.js	—	Shared utilities (readStdin, proceed, block, runHook)

Skills (11 bundled)

Skill	Purpose
session-management	Session lifecycle automation
infrastructure-ops	Container discovery, health checks
parallel-dev	Multi-branch development
orchestration	Multi-phase task decomposition
structured-planning	Guided design workflows
project-lifecycle	Project creation/registration
system-utilities	Git sync, cleanup
upgrade	Self-improvement automation
fabric	AI text processing via Ollama

task-dashboard	Zero-token Pulse task formatting
_template	Template for new skills

Agents (17 total)

Standard Agents (10):

- code-analyzer, code-implementer, code-tester — Development trio
- deep-research — Topic investigation
- docker-deployer — Safe Docker deployment
- service-troubleshooter — Infrastructure diagnosis
- ollama-manager — Local LLM management
- project-plan-validator — Architecture validation
- memory-bank-synchronizer — Context/doc sync
- parallel-dev-{documenter,implementer,tester,validator} — Parallel dev team

Personas (24 in v3.2.0)

Personas are isolated execution profiles for headless jobs, each with explicit permissions, model selection, methodology, and budget constraints.

Safety Tiers:

Tier	Permissions	Personas	Model	Budget Range
Tier 1 (Discovery)	Read-only	investigator	Haiku	\$1.00, 5 turns
Tier 2 (Analyze)	Read + query + label	analyst, task-evaluator, task-investigator, librarian, context-maintainer, creative-feedback, creative-presenter	Sonnet/Haiku	\$2-5, 10-20 turns
Tier 3 (Execute)	Read + write + bash	autofix-executor, bug-fixer, backend-eng, db-eng, ux-eng, infr astructure-deployer, troubleshooter, project-manager, creative-thinker, creative-action, security-reviewer	Sonnet	\$2-15, 20-60 turns
Tier 4 (Complex)	Full reasoning + coordination	ai-reviewer, team-verdict, creative-builder, researcher	Opus	\$3-30, 30-50 turns

Aurora Creative Pipeline Personas (5 dedicated):

- creative-thinker (ideation, 4h), creative-builder (isolated build, daily 2am), creative-presenter (delivery, daily 6am), creative-feedback (reactions, daily 9pm), creative-action (execution, post-feedback)
-

5. Pulse — Shared Task Management

Overview

Pulse is the shared task management backend for Project Aion. It provides a REST API that both Jarvis and Alfred-Pro use to create, track, and coordinate tasks. Built as a custom FastAPI service (our implementation), backed by PostgreSQL on the shared Jarvis infrastructure.

Version: 1.0.0-aion

Port: 8700

Database: pulse (on jarvis-postgres, port 5432)

Service: launchd (com.aion.pulse, 24/7, auto-restart)

API Endpoints

Method	Path	Purpose
GET	/api/v1/health	Health check
GET	/api/v1/tasks	List tasks (filter by status, label, search)
GET	/api/v1/tasks/ready	Ready queue (auto:ready, not blocked)
GET	/api/v1/tasks/{id}	Get task detail
POST	/api/v1/tasks	Create task
PATCH	/api/v1/tasks/{id}	Update task (status, labels, notes, claim)
POST	/api/v1/tasks/{id}/close	Close with reason
POST	/api/v1/tasks/{id}/labels	Add labels
DELETE	/api/v1/tasks/{id}/labels/{label}	Remove label
POST	/api/v1/tasks/{id}/transition	Named transition (approve, claim, pause, complete)
POST	/api/v1/tasks/{id}/stage	Stage transition
POST	/api/v1/triggers/emit	Emit pipeline trigger
POST	/api/v1/triggers/claim-handler	Claim triggers for handler
GET	/api/v1/triggers/pending	Pending triggers
GET/POST	/api/v1/messages	Message bus
PATCH	/api/v1/jobs/{name}	Job state updates

GET/PUT	/api/v1/settings/{key}	Settings CRUD
GET	/api/v1/events	Audit event log
POST	/api/v1/projects/import	Import setup plan (YAML)

Label System

The label system is the heart of Pulse. Labels are organized into two super-groups:

Execution Labels (control pipeline behavior):

Category	Labels	Purpose
auto:	ready, candidate	Automation readiness
risk:	safe, moderate, destructive	Reversibility assessment
pipeline:	evaluated, approved, needs-approval	Pipeline stage
stage:	intake, evaluate, route, review, queue, execute	Task lifecycle stage
waiting:	owner, external, subtasks, session, trigger	Blocking factor
agent:	jarvis, aifred, shared	Archon ownership

Context Labels (metadata, no pipeline effect):

Category	Labels	Purpose
domain:	infrastructure, coding, research, creative, security	Work discipline
project:	chronicler, aifred, jarvis, aion	Owning project
source:	session, headless, claude-code, orchestration	Origin
type:	research, bug, feature, maintenance	Classification
capability:	file-ops, code, research, infrastructure	Required tooling
severity:	critical, high, medium, low	Impact level

Execution Matrix:

- auto:ready + risk:safe → Execute autonomously (no human needed)
- auto:ready + risk:moderate → Individual approval required
- auto:ready + risk:destructive → Manual only
- auto:candidate → Investigation phase

Named Transitions:

- approve: needs-approval → approved + auto:ready + stage:queue
- claim: stage:queue → stage:execute + in_progress
- pause: strip gates → parked + deferred
- complete: strip execution labels → closed
- executor-fail: auto:ready → parked + stage:review

Database Schema

Table	Purpose
tasks	Task records (id, title, description, status, priority, labels[], metadata, notes)
events	Audit trail (task_id, event_type, actor, data, timestamp)
messages	Message bus (event_type, source, severity, data, delivered)
triggers	Pipeline triggers (task_id, stage, handler, status)
job_state	Nexus job tracking (job_name, last_run, fail_count)
settings	Key-value settings store

Jarvis Integration

Jarvis accesses Pulse through the `jarvis-pulse` MCP server:

- 6 tools: `pulse_list`, `pulse_show`, `pulse_create`, `pulse_update`, `pulse_close`, `pulse_stats`
- FastMCP 3.0, HTTP client to localhost:8700
- Label convention: always tag Jarvis tasks with agent:jarvis

6. Shared Infrastructure

Docker Compose Stack

All containers run on the jarvis-net Docker bridge network. Defined in `/Users/nathanielcannon/Claude/Jarvis/infrastructure/docker-compose.yml`.

Container	Image	Port(s)	Purpose	Resources
jarvis-postgres	paradedb/paradedb	5432	PostgreSQL + pgvector + BM25	6GB RAM, 4 CPU
jarvis-qdrant	qdrant/qdrant	6333, 6334	Vector search (embeddings)	8GB RAM, 4 CPU
jarvis-neo4j	neo4j	7474, 7687	Knowledge graph (Graphiti)	6GB RAM, 4 CPU
jarvis-redis	redis/redis-stack	6379, 8001	Cache + working memory	2GB RAM, 2 CPU
jarvis-n8n	n8nio/n8n	5678	Workflow automation	2GB RAM, 2 CPU

PostgreSQL Databases

Database	Owner	Purpose
jarvis	jarvis	Jarvis analytics, session logs, Chronicler data
chronicler	jarvis	Chronicler CDM schema (39 tables, 1.9M records)
n8n	jarvis	n8n workflow engine backend
rag	jarvis	RAG pipeline metadata
pulse	pulse	Pulse task management (Project Aion shared)

Local AI Services

Service	Port	Model	Purpose	Management
MLX Embed	8000	Qwen3-Embedding-4B	Text embeddings (2560-dim)	tmux W6
Ollama	11434	Qwen3:8b, others	Local LLM inference	System service
LiteLLM	4000	Proxy to Ollama	Unified LLM API	tmux W7

Pulse Service

Property	Value
Port	8700
Process Manager	launchd (com.aion.pulse)
Auto-start	Yes (RunAtLoad + KeepAlive)
Log	/Users/nathanielcannon/Claude/AIFred-Pro/.claude/logs/pulse.log
Source	/Users/nathanielcannon/Claude/AIFred-Pro/pulse/app.py

Telegram Bot

Property	Value
Bot	@KeryxArchon_bot
Purpose	Async notifications, approval workflows, watchdog alerts
Config	/Users/nathanielcannon/Claude/AIFred-Pro/.claude/jobs/.env

7. Nexus — Headless Job Orchestration

Overview

Nexus is Alfred-Pro's autonomous job execution framework. It runs Claude Code headless sessions on a schedule, with persona-isolated permissions, cost controls, and human-in-the-loop approval gates.

Single entry point: The dispatcher is the only cron job. Everything triggers through it or Telegram callbacks.

Architecture

```
cron (every 5 min) → dispatcher.sh
  ■■■ Reads registry.yaml (job definitions)
  ■■■ Checks enabled + schedule match
  ■■■ Runs pre-check gates (bash, skip if fails → no AI cost)
  ■■■ Invokes executor.sh for due jobs
      ■■■ Loads persona (prompt + permissions + config)
      ■■■ Resolves engine (claude-code or ollama)
      ■■■ Executes job prompt
      ■■■ Extracts summary + severity
      ■■■ Publishes to message bus → Telegram relay
```

Cron Jobs

Schedule	Script	Purpose
Every 5 min	dispatcher.sh	Master job scheduler
Every 5 min	telegram-callback-handler.sh	Process Telegram responses
Every 15 min	dispatcher-watchdog.sh	Alert if dispatcher stalls

Job Registry

Jobs are defined in `.claude/jobs/registry.yaml`. Each has: description, persona, schedule, budget, pre-check, and prompt.

Job	Schedule	Persona	Purpose
health-check	6h	investigator	Docker container health → infra tasks
task-score	on-demand	task-investigator	Label unlabeled tasks (auto: + risk:)
task-investigator	4h	task-investigator	Promote auto:candidate → auto:ready

task-executor	6h	autofix-executor	Execute auto:ready + risk:safe tasks
weekly-cost-report	Weekly Mon 9am	analyst	Job spending summary
doc-sync-check	24h	investigator	Documentation drift detection
priority-review	Weekly Mon 7am	investigator	Flag stale/stuck tasks

Safety Rails

- 10-task execution cap per run (prevents runaway)
- Git stash before file operations (reversible)
- 3-minute timeout per task execution
- Structured JSON audit trail
- DND-aware notifications (quiet hours: 10 PM-7 AM weekdays, 11 PM-9 AM weekends)
- Relay watchdog (alerts if message delivery stalls)
- Pre-check gates (no AI cost if nothing changed)

Design Principles

1. Single entry point — dispatcher is the only cron job
 2. Persona isolation — explicit permissions per job
 3. Pre-check gates — bash gate before LLM invocation
 4. Human-in-the-loop — critical actions require approval
 5. Label-driven lifecycle — tasks move via labels, not manual status
 6. External watchdog — 15-minute Telegram alerts if dispatcher stalls
 7. Learn, don't repeat — feedback captured and applied
-

8. Integration Layer

Cross-Archon Label Conventions

Label	Meaning
agent:jarvis	Task owned by Jarvis (deep coding, analysis)
agent:aifred	Task owned by Alfred (infrastructure, maintenance)
agent:shared	Either Archon may handle (first responder wins)

Pulse MCP Server

File: /Users/nathanielcannon/Claude/Jarvis/infrastructure/rag-service/pulse_mcp_server.py

Framework: FastMCP 3.0

Transport: stdio (registered in .mcp.json as jarvis-pulse)

Tool	Purpose
pulse_list	List tasks with status/label/search filters
pulse_show	Show task details
pulse_create	Create task (auto-tags agent:jarvis)
pulse_update	Update status, add/remove labels, append notes
pulse_close	Close with reason
pulse_stats	Aggregate statistics

Jarvis Launch Script Integration

The Jarvis tmux launcher (.claude/scripts/launch-jarvis-tmux.sh) includes a pre-flight check for Pulse API health. This is informational (not blocking) — Jarvis can operate without Alfred, but cross-Archon task visibility requires Pulse.

Credential Management

Secret	Location	Purpose
Pulse DB password	.claude/secrets/credentials.yaml (pulse: section)	Jarvis → Pulse DB
Pulse DB password	Alfred-Pro/.env	Pulse server config

Telegram token	AlFred-Pro/.claude/jobs/.env	@KeryxArchon_bot
GitHub PAT	.claude/secrets/credentials.yaml (github: section)	Git push authentication

9. Security Architecture

Jarvis Security

- CLAUDE.md guardrails (prompt-based): never force push main, never store secrets in tracked files, confirmation gates for destructive ops
- Branch protection hook (branch-protection.js): prevents accidental edits to default branch
- Filesystem policy: authorized write locations only, no /tmp or system dirs
- Credential file: .claude/secrets/credentials.yaml (gitignored)

Alfred-Pro Security (3-Layer Guards)

Document Guard (document-guard.js, 27.9KB):

- 4-tier protection: critical, high, medium, low
- 7 check types: no_write, credential scanning, key deletion, section preservation, frontmatter, shebang
- Optional semantic validation via Ollama
- Single-use override mechanism
- Full audit logging

Credential Guard (credential-guard.js, 13.2KB):

- Pattern-based secret detection (AWS, OpenAI, GitHub, Telegram tokens)
- Per-pattern governance policies
- Tool output scanning for pre-leakage

Persona Guard (persona-guard.js, 3.5KB):

- Per-persona tool restrictions
- Discovery/Investigate/Execute/Research/Deploy tiers
- Blocks denied tools before execution

Nexus Safety

- Persona isolation (explicit allow/deny per job)
- Pre-check gates (bash test before LLM cost)
- 10-task execution cap per run
- 3-minute timeout per task
- Git stash before file operations
- DND-aware notifications

10. Operational Procedures

Starting the System

1. Docker Desktop must be running
2. Jarvis tmux session: `bash .claude/scripts/launch-jarvis-tmux.sh`
3. Pulse starts automatically via launchd (`com.aion.pulse`)
4. Nexus dispatcher runs via cron (every 5 min)
5. Telegram notifications via `@KeryxArchon_bot`

Stopping/Restarting Pulse

```
launchctl unload ~/Library/LaunchAgents/com.aion.pulse.plist # Stop
launchctl load ~/Library/LaunchAgents/com.aion.pulse.plist # Start
```

Checking System Health

```
# All infrastructure
docker ps --format "table {{.Names}}\t{{.Status}}"
curl -sf http://localhost:8700/api/v1/health # Pulse
curl -sf http://localhost:6333/healthz # Qdrant
curl -sf http://localhost:7474 # Neo4j

# Nexus
crontab -l | grep dispatcher # Cron entries
tail -20 ~/Claude/AIFred-Pro/.claude/logs/headless/dispatcher.log # Dispatcher log
```

Creating Cross-Archon Tasks

From Jarvis (via MCP):

Use `pulse_create` tool with labels including `agent:jarvis`

From Alfred (via CLI/API):

```
pulse create "Task title" -l agent:aifred -l domain:infrastructure
```

From command line:

```
curl -X POST http://localhost:8700/api/v1/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Task","labels":["agent:shared"]}'
```

Key File Locations

Purpose	Path
Jarvis session state	/Users/nathanielcannon/Claude/Jarvis/.claude/context/session-state.md
Jarvis scratchpad	/Users/nathanielcannon/Claude/Jarvis/.claude/context/scratchpad.md
Jarvis credentials	/Users/nathanielcannon/Claude/Jarvis/.claude/secrets/credentials.yaml
Jarvis MCP config	/Users/nathanielcannon/Claude/Jarvis/.mcp.json
Jarvis Docker	/Users/nathanielcannon/Claude/Jarvis/infrastructure/docker-compose.yml
Alfred-Pro config	/Users/nathanielcannon/Claude/Alfred-Pro/.env
Alfred-Pro job registry	/Users/nathanielcannon/Claude/Alfred-Pro/.claude/jobs/registry.yaml
Alfred-Pro Telegram	/Users/nathanielcannon/Claude/Alfred-Pro/.claude/jobs/.env
Pulse source	/Users/nathanielcannon/Claude/Alfred-Pro/pulse/app.py
Pulse launchd	~/Library/LaunchAgents/com.aion.pulse.plist
Pulse log	/Users/nathanielcannon/Claude/Alfred-Pro/.claude/logs/pulse.log
Pulse MCP	/Users/nathanielcannon/Claude/Jarvis/infrastructure/rag-service/pulse_mcp_server.py